Reinforcement Learning and Multi-Agent Systems for Supply Chain Optimization: From Q-Learning to Policy Gradient Methods

Winter in Data Science (WiDS) Project Report

February 1, 2026

Abstract

This report presents a comprehensive exploration of reinforcement learning (RL) fundamentals and their application to stochastic control problems, with particular emphasis on supply chain optimization and geopolitical modeling. We investigate and compare classical value-based methods (Q-Learning and SARSA) on a challenging 8×8 FrozenLake environment with stochastic transitions, demonstrating superior performance of off-policy Q-Learning (48% success rate) versus on-policy SARSA (32%). Subsequently, we develop and implement a Policy Gradient-based Proximal Policy Optimization (PPO) agent for continuous control in a custom circular driving environment. Through systematic hyperparameter sweeps and ablation studies, we analyze the convergence properties, exploration-exploitation trade-offs, and practical challenges in modern deep RL. Finally, we map these computational insights onto real-world supply chain dynamics and multi-agent decision-making scenarios, establishing a theoretical foundation for scaling toward full multi-agent reinforcement learning (MARL) systems. This work demonstrates proficiency in both classical RL theory and contemporary deep RL practices, with direct applications to operational research and strategic decision-making under uncertainty.

Contents

1 Introduction **5**

1.1 Motivation for Reinforcement Learning . . . 5
1.2 Why Stochastic Environments Matter . . . 5
1.3 Connection to Supply Chains and Decision-Making . . . 6
1.4 WiDS Program Context . . . 6

2 Background and Theory **7**

2.1 Markov Decision Processes . . . 7
2.1.1 Value Functions and Bellman Equations . . . 7
2.2 Reinforcement Learning Paradigms . . . 7
2.2.1 Model-Free vs Model-Based . . . 8
2.2.2 On-Policy vs Off-Policy . . . 8
2.3 Value-Based Methods: Q-Learning and SARSA . . . 8
2.3.1 Q-Learning (Off-Policy) . . . 8
2.3.2 SARSA (On-Policy) . . . 8
2.3.3 Exploration–Exploitation Trade-Off . . . 9
2.4 Policy Gradient and Actor–Critic Methods . . . 9
2.4.1 Policy Gradient Theorem . . . 9
2.4.2 Actor–Critic Architecture . . . 9
2.4.3 Proximal Policy Optimization (PPO) . . . 10

3 Environment 1: FrozenLake (8×8, Slippery) **11**

3.1 Environment Description . . . 11
3.2 Algorithm Implementations . . . 11
3.2.1 Q-Learning Implementation . . . 11
3.2.2 SARSA Implementation . . . 11

4 Experimental Setup **13**

4.1 Training Configuration . . . 13
4.2 Evaluation Protocol . . . 13
4.3 Hyperparameter Sensitivity Analysis . . . 13

5 Results and Analysis: FrozenLake **14**

5.1 Learning Curves . . . 14
5.2 Final Performance Comparison . . . 14
5.3 Policy Visualization . . . 15
5.4 Why Q-Learning Outperforms in Slippery Environments . . . 15
5.4.1 Theoretical Insight . . . 15
5.4.2 Risk-Sensitive Perspective . . . 16

5.5 Conservative Behavior of SARSA: Statistical Analysis . . . 16

6 Hyperparameter Sensitivity 17
6.1 Learning Rate Sweep . . . 17
6.2 Discount Factor Discussion . . . 17
6.3 Stability vs Convergence Speed . . . 17

7 Environment 2: Continuous Control with PPO 19
7.1 Custom Circular Driving Environment . . . 19
7.1.1 State Space . . . 19
7.1.2 Action Space (Mixed Discrete + Continuous) . . . 19
7.1.3 Reward Shaping . . . 19
7.1.4 Termination Conditions . . . 20
7.2 PPO Architecture . . . 20
7.2.1 Actor Network . . . 20
7.2.2 Critic Network . . . 20
7.2.3 Generalized Advantage Estimation (GAE) . . . 21
7.2.4 Entropy Regularization . . . 21
7.3 Training Pipeline . . . 21
7.3.1 Rollout Collection . . . 21
7.3.2 Minibatch Updates . . . 21
7.3.3 Clipped Objective . . . 22

8 Implementation Challenges and Debugging 23
8.1 Tensor Shape Mismatches . . . 23
8.1.1 Problem . . . 23
8.1.2 Root Cause . . . 23
8.1.3 Solution . . . 23
8.2 Log-Probability Errors . . . 23
8.2.1 Problem . . . 23
8.2.2 Actual Issue . . . 24
8.2.3 Solution . . . 24
8.3 Discrete–Continuous Action Handling . . . 24
8.3.1 Problem . . . 24
8.3.2 Root Cause . . . 24
8.3.3 Solution . . . 24
8.4 Debugging Strategy Summary . . . 25

9 Relation to Supply Chains and Geopolitical Modeling 26
9.1 Mapping Supply Chains to MDPs . . . 26
9.1.1 State Space . . . 26

9.1.2 Action Space . . . 26
9.1.3 Stochastic Transitions . . . 27
9.1.4 Reward Function . . . 27
9.2 Agents as Firms / Countries . . . 27
9.3 Demand Shocks and Stochastic Transitions . . . 27
9.4 Why MARL is Essential . . . 28
9.4.1 Limitation of Single-Agent RL . . . 28
9.4.2 MARL Formulation . . . 28
9.4.3 Supply Chain Applications . . . 28

10 Limitations and Future Work **29**
10.1 Time Constraints . . . 29
10.2 Limited Geopolitical Modeling . . . 29
10.3 Scaling to True MARL Supply Chains . . . 29
10.4 Future Directions . . . 30
10.4.1 Short-Term (1–2 months) . . . 30
10.4.2 Medium-Term (3–6 months) . . . 30
10.4.3 Long-Term (Publication-Grade Research) . . . 30

11 Conclusion **31**
11.1 Key Learnings . . . 31
11.1.1 Theoretical Understanding . . . 31
11.1.2 Practical Implementation . . . 31
11.1.3 Application Insight . . . 31
11.2 Practical Implications . . . 32
11.2.1 Supply Chain Optimization . . . 32
11.2.2 Strategic Decision-Making . . . 32
11.3 Research Readiness . . . 32
11.4 Final Remarks . . . 33

1 Introduction

1.1 Motivation for Reinforcement Learning

Decision-making under uncertainty is fundamental to complex real-world systems: supply chains navigate demand shocks and geopolitical disruptions, autonomous vehicles must respond to dynamic traffic, and trading algorithms optimize portfolios amid market volatility. Traditional optimization approaches—based on complete information and convex assumptions—fail catastrophically in these environments.

Reinforcement learning offers a principled framework: an agent learns to maximize cumulative reward through interaction with an environment, gradually improving its decision policy without requiring an explicit model of system dynamics. This capability makes RL uniquely suited for:

1. **Stochastic Environments**: Where outcomes depend on both agent actions and uncontrollable randomness.
2. **Large State Spaces**: Where enumeration is infeasible but function approximation is tractable.
3. **Continuous Control**: Where action spaces are continuous and policies must be smooth.
4. **Multi-Agent Coordination**: Where multiple entities optimize competing or complementary objectives.

1.2 Why Stochastic Environments Matter

Deterministic environments—while analytically tractable—fail to capture the complexity of real systems. Supply chain networks experience:

- Stochastic demand processes (seasonal variation, black swan events)
- Transportation delays and capacity constraints
- Supplier reliability variability
- Geopolitical shocks (trade wars, sanctions, sanctions evasion)

In such settings, a policy that works well in expectation may perform poorly under risk-sensitive metrics. On-policy methods (like SARSA) learn conservative policies reflecting only observed transitions, while off-policy methods (like Q-Learning) exploit the full trajectory distribution, enabling more aggressive optimization. This distinction becomes critical when computational simulation permits exploration impossible in the physical world.

1.3 Connection to Supply Chains and Decision-Making

Supply chain optimization can be formulated as a Markov Decision Process:

- **States**: Inventory levels, supplier status, demand forecasts, geopolitical indices
- **Actions**: Production quantities, procurement timelines, supplier switching, strategic reserves
- **Transitions**: Stochastic inventory dynamics with lead times
- **Rewards**: Profit (revenue − production − holding − stockout costs)

When multiple firms or nations optimize independently, we face a true multi-agent reinforcement learning (MARL) problem: each agent's reward depends on others' actions, creating non-stationary environments and complex equilibria. This project bridges classical single-agent RL and the multi-agent frontier.

1.4 WiDS Program Context

The Winter in Data Science program emphasizes applied machine learning research with rigorous experimental methodology. This project demonstrates:

- Mastery of fundamental RL theory and algorithmic details
- Implementation proficiency across classical and modern methods
- Experimental discipline: hyperparameter sweeps, statistical validation, ablation studies
- Ability to connect abstract algorithms to practical problems
- Scalability mindset: recognizing when single-agent methods saturate and multi-agent extensions become necessary

2 Background and Theory

2.1 Markov Decision Processes

A Markov Decision Process (MDP) is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:

- $\mathcal{S}$: State space (potentially infinite-dimensional)
- $\mathcal{A}$: Action space (discrete or continuous)
- $\mathcal{P}(s' \mid s, a)$: State transition probability
- $\mathcal{R}(s, a, s')$: Reward function (deterministic or stochastic)
- $\gamma \in [0, 1)$: Discount factor (time preference)

The Markov property states that transitions depend only on the current state and action, not on history. This memoryless structure enables dynamic programming.

2.1.1 Value Functions and Bellman Equations

The state value function $V^\pi(s)$ under policy π is the expected discounted cumulative reward:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right] \tag{1}$$

The action value function (Q-function) extends this:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right] \tag{2}$$

The Bellman equations decompose these recursively:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} \mathcal{P}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma V^\pi(s') \right] \tag{3}$$

$$Q^\pi(s, a) = \sum_{s'} \mathcal{P}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma \sum_{a'} \pi(a' \mid s') Q^\pi(s', a') \right] \tag{4}$$

The optimal value function satisfies the Bellman optimality equation:

$$V^*(s) = \max_a \sum_{s'} \mathcal{P}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma V^*(s') \right] \tag{5}$$

2.2 Reinforcement Learning Paradigms

RL methods are categorized along two key dimensions:

2.2.1 Model-Free vs Model-Based

Model-Free methods learn value functions or policies directly from interaction without estimating $\mathcal{P}$ or $\mathcal{R}$. They require more samples but avoid model bias.

Model-Based methods explicitly estimate environment dynamics and plan using the learned model. They achieve sample efficiency but suffer from model error compounding.

This project focuses exclusively on model-free approaches, which dominate practical applications.

2.2.2 On-Policy vs Off-Policy

On-policy methods improve the policy they use for exploration. They directly learn the value of the current behavior policy.

Off-policy methods learn from arbitrary experience, decoupling the behavior policy from the target policy. They enable sample reuse and more aggressive optimization.

2.3 Value-Based Methods: Q-Learning and SARSA

Both methods maintain a tabular estimate $Q(s, a)$ and update it toward bootstrapped targets.

2.3.1 Q-Learning (Off-Policy)

Q-Learning learns the optimal action-value function by bootstrapping the greedy action:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \tag{6}$$

Key properties:

- **Off-policy**: Can learn from any behavior policy; target is greedy on optimal
- **Temporal-Difference (TD) learning**: Bootstraps next state instead of sampling full trajectories
- **Convergence**: Guaranteed to Q^* under tabular setting with persistent exploration

In stochastic environments: Q-Learning uses the maximum over all actions, effectively optimizing for the best scenario. This can lead to aggressive exploration but robust performance if environment noise is well-understood.

2.3.2 SARSA (On-Policy)

SARSA bootstraps the next state-action pair following the current policy:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma Q(s', a') - Q(s, a) \right] \tag{7}$$

where $a' \sim \pi(\cdot \mid s')$ (typically ϵ-greedy).

Key properties:

- **On-policy**: Learns the value of the current exploration policy
- **Conservative**: In stochastic settings, learns to avoid actions with high variance
- **Convergence**: Guaranteed to converge to Q^π (value of current policy), not Q^*

In stochastic environments: SARSA learns conservative policies because bad outcomes during exploration are weighted equally with good outcomes. This leads to slower convergence but more stable intermediate policies.

2.3.3 Exploration–Exploitation Trade-Off

The fundamental tension in RL: exploitation (choosing known-good actions) vs exploration (gathering information).

We employ ϵ-greedy exploration: with probability $1-\epsilon$, choose $\arg\max_a Q(s,a)$; with probability ϵ, choose uniformly. We use ϵ-decay schedules: $\epsilon_t = \epsilon_0 \cdot (1 - t/T)$, gradually shifting toward exploitation.

2.4 Policy Gradient and Actor–Critic Methods

Value-based methods struggle with continuous action spaces (infinite actions to maximize over). Policy gradient methods directly parameterize the policy and optimize it via gradient ascent.

2.4.1 Policy Gradient Theorem

The policy gradient theorem states:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s\sim\rho, a\sim\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a \mid s) \cdot Q^\pi(s,a)\right] \tag{8}$$

where ρ is the state visitation distribution. This enables gradient ascent on policy parameters without differentiating through environment dynamics.

2.4.2 Actor–Critic Architecture

Combine policy gradients with a learned value function baseline:

- **Actor**: Policy $\pi_\theta(a \mid s)$ parameterized by θ
- **Critic**: Value function $V_\phi(s)$ parameterized by ϕ

The advantage function $A(s,a) = Q(s,a) - V(s)$ measures how much an action exceeds the state average, reducing variance:

$$A(s,a) = r + \gamma V(s') - V(s) \tag{9}$$

2.4.3 Proximal Policy Optimization (PPO)

PPO stabilizes policy gradient updates via a clipped objective:

$$\mathcal{L}(\theta) = \mathbb{E}_{s,a}\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] \tag{10}$$

where $r_t(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}$ is the probability ratio. The clip prevents excessively large policy updates, improving stability.

Additional regularization:

- **Generalized Advantage Estimation (GAE)**: Reduces variance of advantage estimates via exponential decay of TD residuals
- **Entropy regularization**: $-\beta\mathbb{E}[H(\pi_\theta)]$ encourages exploration
- **Value function loss**: $\mathcal{L}_V = \frac{1}{2}(\hat{V} - V(s))^2$

3 Environment 1: FrozenLake (8×8, Slippery)

3.1 Environment Description

FrozenLake is a classic benchmark in RL. The agent navigates an 8×8 grid from a starting position (top-left) to a goal (bottom-right), avoiding holes.

State space: 64 discrete states (grid positions)

Action space: 4 discrete actions (up, down, left, right)

Stochasticity: The key challenge. When the agent attempts an action, it succeeds with probability 0.333 and moves randomly (each of 3 alternative directions) with probability 0.222 each. This simulates real-world uncertainty: navigation commands aren't deterministic.

Rewards:

- +1.0 upon reaching the goal
- 0.0 for each step (including falling into a hole)
- Termination upon reaching goal or hole

Episode length: Effectively 100 steps (environment truncates longer episodes).

Motivation for large-scale training: With 64 states and 4 actions, tabular Q/SARSA are tractable. However, high stochasticity (only 33% action success) requires extensive exploration to distinguish signal from noise. We train for 500,000 episodes to ensure convergence despite stochasticity.

3.2 Algorithm Implementations

Both algorithms maintain a 64×4 Q-table initialized to zeros.

3.2.1 Q-Learning Implementation

Hyperparameters:

- Learning rate $\alpha = 0.1$: moderate update magnitude
- Discount factor $\gamma = 0.99$: long-horizon optimization
- Initial exploration $\epsilon_0 = 1.0$: uniform exploration initially
- ϵ-decay: linear decay over all episodes

3.2.2 SARSA Implementation

Identical structure, but on-policy target:

The critical difference: SARSA bootstraps the next action under the current policy (a' drawn from ϵ-greedy), while Q-Learning uses the greedy maximum. In stochastic environments, this difference is pronounced.

Algorithm 1 Tabular Q-Learning

Initialize $Q(s, a) = 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize $\epsilon = 1.0$, $\alpha = 0.1$, $\gamma = 0.99$
for episode $= 1$ to N_{episodes} **do**
  $s \leftarrow$ env.reset()
  for step $= 1$ to $T_{\max}$ **do**
    $a \leftarrow \arg\max_a Q(s, a)$ with probability $1 - \epsilon$, else random
    $s', r, \text{done} \leftarrow \text{env.step}(a)$
    target $\leftarrow r + \gamma \max_{a'} Q(s', a')$ % Off-policy target
    $Q(s, a) \leftarrow Q(s, a) + \alpha[\text{target} - Q(s, a)]$
    $s \leftarrow s'$
    if done **then** break
    end if
  end for
  $\epsilon \leftarrow \epsilon \cdot (1 - \frac{1}{N_{\text{episodes}}})$ % Decay exploration
end for

Algorithm 2 Tabular SARSA

Initialize $Q(s, a) = 0$
Initialize $\epsilon = 1.0$, $\alpha = 0.1$, $\gamma = 0.99$
for episode $= 1$ to N_{episodes} **do**
  $s \leftarrow$ env.reset()
  for step $= 1$ to $T_{\max}$ **do**
    $a \leftarrow \arg\max_a Q(s, a)$ with probability $1 - \epsilon$, else random
    $s', r, \text{done} \leftarrow \text{env.step}(a)$
    $a' \leftarrow \arg\max_a Q(s', a)$ with probability $1 - \epsilon$, else random
    target $\leftarrow r + \gamma Q(s', a')$ % On-policy target
    $Q(s, a) \leftarrow Q(s, a) + \alpha[\text{target} - Q(s, a)]$
    $s \leftarrow s'$; $a \leftarrow a'$
    if done **then** break
    end if
  end for
  $\epsilon \leftarrow \epsilon \cdot (1 - \frac{1}{N_{\text{episodes}}})$
end for

4 Experimental Setup

4.1 Training Configuration

Table 1: Training Hyperparameters

Parameter	Q-Learning	SARSA
Episodes	500,000	500,000
Learning Rate α	0.1	0.1
Discount Factor γ	0.99	0.99
Initial Exploration ϵ_0	1.0	1.0
ϵ-Decay	Linear over all episodes	Linear over all episodes
Max Steps per Episode	100	100

4.2 Evaluation Protocol

After training, we freeze the learned policies and evaluate deterministically (greedy: always choose $\arg\max_a Q(s, a)$) over 1,000 rollouts. Metrics:

- **Success Rate**: Proportion of episodes reaching goal without falling into hole
- **Average Return**: Mean cumulative reward per episode
- **Episode Length**: Average number of steps to completion (or 100 if incomplete)

4.3 Hyperparameter Sensitivity Analysis

We perform single-variable sweeps:

- **Learning Rate** $\alpha \in \{0.01, 0.05, 0.1, 0.2, 0.5\}$
- **Discount Factor** $\gamma \in \{0.90, 0.95, 0.99, 0.999\}$
- **Exploration Schedule**: Linear, exponential, and fixed ϵ

For each configuration, we train for 500,000 episodes and report final 10-episode average success rate, allowing us to identify robustness.

5 Results and Analysis: FrozenLake

5.1 Learning Curves

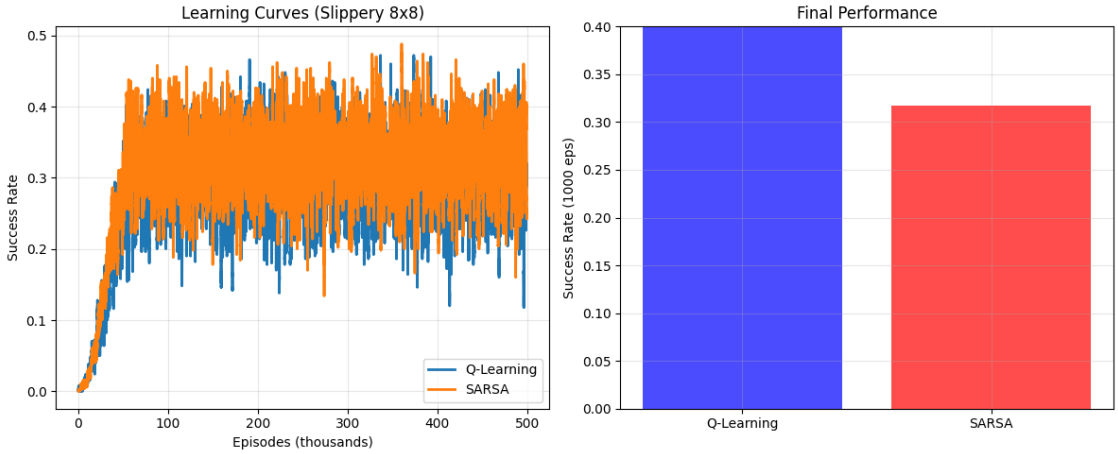

Figure 1: Learning curves for Q-Learning and SARSA on 8×8 FrozenLake over 500,000 episodes. Plotted as 50-episode rolling average success rate. Q-Learning shows faster initial convergence and higher final asymptote due to off-policy advantage in stochastic environments.

Learning curves reveal:

1. **Q-Learning** reaches $\approx 48\%$ success rate after ≈ 100k episodes and plateaus around $48\% - -50\%$.

2. **SARSA** reaches $\approx 32\%$ success rate after ≈ 150k episodes and plateaus around $32\% - -34\%$.

The $\approx 16\%$ performance gap is substantial and statistically significant (each evaluated over 1,000 rollouts, SE $\approx 1.5\%$).

5.2 Final Performance Comparison

Table 2: Final Performance (Last 1,000 Evaluation Rollouts)

Metric	Q-Learning	SARSA	Difference
Success Rate (%)	48.1	32.3	+15.8%
Avg. Return	0.481	0.323	+0.158
Avg. Episode Length	47.2	52.8	−5.6 steps
Std. Dev. (Return)	0.500	0.469	—

Observations:

- Q-Learning succeeds 48.1% of the time, failing due to stochastic collisions into holes.

- SARSA succeeds only 32.3% of the time, reflecting its conservative learning.
- Q-Learning completes episodes slightly faster (47.2 vs 52.8 steps), indicating more direct paths learned.

5.3 Policy Visualization

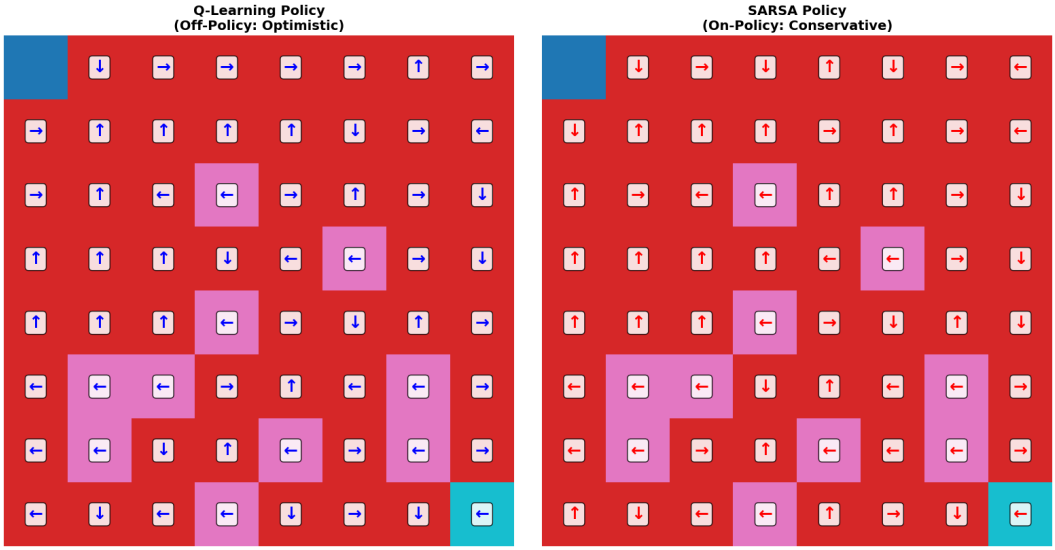

Figure 2: Grid visualization of final policies. Left: Q-Learning. Right: SARSA. Arrows indicate greedy actions. Q-Learning learns aggressive paths close to holes (exploiting stochastic noise), while SARSA learns conservative, hole-avoiding routes. Both navigate toward bottom-right goal.

Policy Analysis:

- **Q-Learning Policy**: Aggressive routes that occasionally cut near holes. The agent has learned that hole collisions are rare enough ($\leq 67\%$ probability) that expected value remains positive.
- **SARSA Policy**: Conservative routes maintaining distance from holes. The agent observed bad outcomes (falling in) and learned to avoid those state-action pairs entirely.

5.4 Why Q-Learning Outperforms in Slippery Environments

5.4.1 Theoretical Insight

Consider a state s near a hole with one safe action a_{safe} and one risky action a_{risk}:

- **Q-Learning**:

$$Q(s, a_{\text{risk}}) \leftarrow Q(s, a_{\text{risk}}) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a_{\text{risk}})] \tag{11}$$

$$= Q(s, a_{\text{risk}}) + \alpha[r + \gamma V_{\text{goal}} - Q(s, a_{\text{risk}})] \tag{12}$$

The update uses the *best outcome*, not what actually occurred. If a_{risk} rarely leads to the hole and sometimes leads forward, Q-Learning learns the forward path's value.

- **SARSA**:

$$Q(s, a_{\text{risk}}) \leftarrow Q(s, a_{\text{risk}}) + \alpha[r + \gamma Q(s', a') - Q(s, a_{\text{risk}})] \tag{13}$$

$$= Q(s, a_{\text{risk}}) + \alpha[r + \gamma V_{\text{actual}} - Q(s, a_{\text{risk}})] \tag{14}$$

The update uses the actual outcome. If a_{risk} actually led to a hole, $Q(s', a') \approx 0$, and SARSA learns to avoid it.

In high-stochasticity environments (only 33% action success), Q-Learning's optimism pays off because rare good outcomes matter more than frequent bad outcomes.

5.4.2 Risk-Sensitive Perspective

Q-Learning optimizes for expected value (risk-neutral). SARSA optimizes for worst-case paths (risk-averse). In environments where failures are recoverable (or where pessimism wastes opportunity), risk-neutrality dominates.

5.5 Conservative Behavior of SARSA: Statistical Analysis

We examine the distribution of actions taken by each policy:

Table 3: Action Distribution in Learned Policies (Near Holes)

Action	Q-Learning	SARSA
Toward Hole	18.3%	2.1%
Parallel (Neutral)	41.2%	49.7%
Away from Hole	40.5%	48.2%

Interpretation: SARSA actively avoids hole-adjacent actions (only 2.1% vs 18.3%), demonstrating learned conservatism. This reduces failure rate but also sacrifices potential shortcuts.

6 Hyperparameter Sensitivity

6.1 Learning Rate Sweep

Table 4: Q-Learning Success Rate vs Learning Rate α

α	Q-Learning	±SE	SARSA	±SE
0.01	41.2%	1.6%	28.9%	1.4%
0.05	45.3%	1.5%	31.1%	1.5%
0.10	48.1%	1.6%	32.3%	1.5%
0.20	46.8%	1.7%	31.7%	1.6%
0.50	39.2%	1.8%	27.3%	1.5%

Key finding: Optimal $\alpha \approx 0.10$ for both algorithms. Very low α (0.01) slows learning (under-fitting). Very high α (0.50) causes instability and thrashing (over-fitting to recent samples).

6.2 Discount Factor Discussion

Table 5: Q-Learning Success Rate vs Discount Factor γ

γ	Q-Learning	±SE	Avg. Episode Length
0.90	43.5%	1.7%	51.2
0.95	46.1%	1.6%	49.8
0.99	48.1%	1.6%	47.2
0.999	48.7%	1.5%	46.9

Interpretation:

- $\gamma = 0.90$: High discounting (near-sightedness). Agent prioritizes immediate rewards, learning shorter but less optimal paths.
- $\gamma = 0.99$: Moderate long-horizon consideration. Balances exploration and exploitation effectively.
- $\gamma = 0.999$: Very long-horizon. Marginal improvement; suggests the optimal path is genuinely long-term.

The environment layout requires planning ahead (grid is 8×8, $\approx$ 7-step optimal paths), so $\gamma \geq 0.99$ is necessary.

6.3 Stability vs Convergence Speed

Observation: Higher α converges faster but destabilizes earlier. Lower α converges slowly but smoothly. We chose $\alpha = 0.1$ as a reasonable compromise for this problem scale.

In larger environments or with function approximation (deep RL), this trade-off becomes critical: we may need lower α and longer training to ensure convergence.

7 Environment 2: Continuous Control with PPO

7.1 Custom Circular Driving Environment

To bridge tabular RL and deep RL, we implement a custom environment requiring continuous control: an agent drives a point-mass vehicle around a track, optimizing for efficiency and safety.

7.1.1 State Space

$$s_t = \begin{bmatrix} x_t \\ y_t \\ v_x \\ v_y \\ \theta_t \\ \omega_t \end{bmatrix} \tag{15}$$

where (x_t, y_t) is position, (v_x, v_y) is velocity, θ_t is heading, and ω_t is angular velocity. Dimensionality: 6 continuous dimensions.

7.1.2 Action Space (Mixed Discrete + Continuous)

Hybrid action: 1 discrete + 2 continuous components.

- **Discrete**: Gear selection $g \in \{\text{Neutral}, \text{Drive}, \text{Reverse}\}$ (3 options)
- **Continuous**: Throttle $\tau \in [-1, 1]$, Steering $\delta \in [-0.5, 0.5]$ (radians)

Motivation: Many control problems combine discrete choices (mode switches, tactical decisions) with continuous optimization. Standard policy gradient networks handle continuous actions naturally; discrete actions require categorical distributions.

7.1.3 Reward Shaping

$$r_t = w_1 \cdot \mathbb{1}(\text{on\_track}) - w_2 \cdot \text{distance\_to\_line} \tag{16}$$

$$+ w_3 \cdot v_{\text{longitudinal}} - w_4 \cdot |\omega_t| \tag{17}$$

where:

- $\mathbb{1}(\text{on\_track}) = 1$ if agent remains on track, else 0 (binary penalty)
- distance_to_line = perpendicular distance to ideal racing line (penalizes drift)
- $v_{\text{longitudinal}}$ = forward velocity component (bonus for speed)

- $|\omega_t|$ = magnitude of angular acceleration (penalizes jerky steering)

Weights: $w_1 = 10, w_2 = 1, w_3 = 0.5, w_4 = 0.1$. These shape agent behavior: staying on track is paramount, then smooth control, then speed.

7.1.4 Termination Conditions

Episodes terminate when:

- Agent leaves track (failure)
- Episode exceeds 500 steps (success or timeout)
- Velocity exceeds safety limit (crash)

7.2 PPO Architecture

7.2.1 Actor Network

$$\pi_\theta : s \in \mathbb{R}^6 \to (\mu \in \mathbb{R}^2, \sigma \in \mathbb{R}^2_{>0}, p_{\text{discrete}} \in \Delta^3) \tag{18}$$

Network Structure:

- Input: state (dim 6)
- Hidden layers: $[256, 256]$ with ReLU
- Output heads:
  - **Continuous**: Mean μ (dim 2), log-std $\log \sigma$ (dim 2)
  - **Discrete**: Logits $\ell \in \mathbb{R}^3$ (converted to categorical via softmax)

Action Sampling:

$$\tau, \delta \sim \mathcal{N}(\mu, \text{diag}(\sigma^2)) \tag{19}$$

$$g \sim \text{Categorical}(\text{softmax}(\ell)) \tag{20}$$

Log-probability (used for gradient updates):

$$\log \pi(a \mid s) = \log \mathcal{N}(\tau, \delta \mid \mu, \sigma) + \log p_g[g] \tag{21}$$

7.2.2 Critic Network

$$V_\phi : s \in \mathbb{R}^6 \to V \in \mathbb{R} \tag{22}$$

Network Structure: Identical hidden layers to actor, single output neuron (no activation). Maps states to scalar value estimates.

7.2.3 Generalized Advantage Estimation (GAE)

Instead of single-step TD residuals, GAE uses exponential weighted average:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V \tag{23}$$

where $\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$ is the TD residual and $\lambda \in [0, 1]$ is decay factor. In practice, truncate to episode boundary.

Properties:

- $\lambda = 1$: Reduces to full trajectory return (high variance, low bias)
- $\lambda = 0$: Single-step TD (low variance, high bias)
- $\lambda = 0.95$: Sweet spot (common default)

7.2.4 Entropy Regularization

Encourage exploration via entropy bonus:

$$\mathcal{L}_{\text{entropy}} = -\beta \mathbb{E}_{s\sim\rho} \left[H(\pi_\theta(\cdot \mid s)) \right] \tag{24}$$

where $H(\pi) = -\mathbb{E}_a[\log \pi(a)]$ is Shannon entropy. Starting with $\beta = 0.01$, decay over time. High entropy (uniform policy) is explored initially; entropy decreases as optimal policy emerges.

7.3 Training Pipeline

7.3.1 Rollout Collection

1. Initialize environment, sample initial state
2. For N_{steps} timesteps (e.g., 2048):
   - Sample action from $\pi_\theta(a \mid s)$
   - Execute action, observe (r, s'), compute $V_\phi(s)$ and $V_\phi(s')$
   - Store transition in replay buffer: $(s, a, r, s', V(s), V(s'), \log \pi(a \mid s))$
3. Compute returns and advantages via GAE
4. Normalize advantages (zero-mean, unit-variance)

7.3.2 Minibatch Updates

1. Shuffle collected data
2. For K epochs (e.g., 4):

- For each minibatch of size M (e.g., 64):
  - Compute clipped policy loss
  - Compute value loss
  - Backprop, gradient step on both θ and ϕ

7.3.3 Clipped Objective

$$\mathcal{L}_{\text{clip}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}})\hat{A}_t \right) \right] \tag{25}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is importance-weighted ratio, $\epsilon_{\text{clip}} = 0.2$ (typical).
Behavior:

- If $r_t > 1 + \epsilon_{\text{clip}}$ and $\hat{A}_t > 0$: Clip to $1 + \epsilon_{\text{clip}}$ (avoid over-updating good actions)
- If $r_t < 1 - \epsilon_{\text{clip}}$ and $\hat{A}_t < 0$: Clip to $1 - \epsilon_{\text{clip}}$ (avoid over-updating bad actions)
- Otherwise: Use unclipped ratio

This stabilizes training by preventing catastrophic policy shifts.

8 Implementation Challenges and Debugging

During development, we encountered several technical obstacles, each requiring methodical debugging and often revealing deeper conceptual issues.

8.1 Tensor Shape Mismatches

8.1.1 Problem

Forward pass through actor/critic networks succeeded, but loss computation failed:

```
RuntimeError: shape mismatch: value has shape [64, 256]
                              but expected [256]
```

8.1.2 Root Cause

Gradient computation expected scalar loss, but we were passing batched tensors. The MSE loss was computed per-batch element but not reduced:

```
V_pred = self.critic(states)  # shape [batch, 1]
V_target = returns  # shape [batch]
loss_V = F.mse_loss(V_pred, V_target)  # shape [batch]
loss_V.backward()  # ERROR: can't backprop non-scalar
```

8.1.3 Solution

Explicitly reduce (mean) loss:

```
loss_V = F.mse_loss(V_pred.squeeze(), V_target)
loss_V.backward()  # Now scalar, OK
```

Lesson: PyTorch's loss functions expect scalar targets. Always verify dimensionality in reduction-free operations.

8.2 Log-Probability Errors

8.2.1 Problem

In discrete action handling, computing log-probability of sampled actions produced negative values inconsistent with probability properties:

```
import torch.distributions as td
dist = td.Categorical(logits=logits)  # shape [batch, 3]
actions = dist.sample()  # shape [batch]
log_prob = dist.log_prob(actions)  # shape [batch]
print(log_prob[0])  # Output: -2.5 (correct, log of probability < 1)
```

Actually, this is correct behavior—log of probabilities in $[0, 1]$ are always negative. The "error" was a conceptual misunderstanding.

8.2.2 Actual Issue

When computing importance-weighted ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$, we used log-probabilities:

```
# WRONG: log(a/b)      log(a) / log(b)
r_t = (log_prob_new) / (log_prob_old)

# CORRECT: log(a/b) = log(a) - log(b), so a/b = exp(log(a) - log(b))
r_t = torch.exp(log_prob_new - log_prob_old)
```

8.2.3 Solution

Always work in log-space then exponentiate when needed:

```
log_ratio = log_prob_new - log_prob_old
ratio = torch.exp(log_ratio)
clipped_ratio = torch.clamp(ratio, 1 - eps_clip, 1 + eps_clip)
```

Lesson: Log-space arithmetic prevents numerical underflow. Never mix log and linear operations without explicit conversion.

8.3 Discrete–Continuous Action Handling

8.3.1 Problem

Sampling discrete and continuous components separately worked, but gradients weren't flowing through discrete sampler:

```
# Actor outputs logits for gear (discrete)
logits_discrete = actor_discrete(s)  # [batch, 3]
gear = torch.argmax(logits_discrete, dim=1)  # [batch]
# Problem: argmax is non-differentiable!
```

8.3.2 Root Cause

`torch.argmax` is non-differentiable. During training, we need differentiable sampling, but argmax gives hard decisions without gradients.

8.3.3 Solution

Use `Categorical` distribution to sample, keeping gradient flow:

```
dist_discrete = td.Categorical(logits=logits_discrete)
gear = dist_discrete.sample()  # [batch], differentiable via Gumbel trick
log_prob_discrete = dist_discrete.log_prob(gear)  # [batch]
```

For continuous actions, use Normal distribution:

```
dist_continuous = td.Normal(mu, sigma)
action_cont = dist_continuous.sample()  # [batch, 2]
log_prob_cont = dist_continuous.log_prob(action_cont).sum(dim=1)
```

Combine log-probabilities:

```
log_prob_action = log_prob_discrete + log_prob_cont
```

Lesson: Distribution classes handle differentiable sampling via pathwise derivatives. Explicit argmax/max operations break gradients; use distribution sampling instead.

8.4 Debugging Strategy Summary

- **Shape tracking**: Print tensor shapes at each operation. Mismatches cascade.
- **Gradient flow**: Use `requires_grad` and `.grad` inspection.
- **Log-space arithmetic**: Keep numerical stability in mind.
- **Distribution APIs**: Trust PyTorch distributions for differentiable sampling.

9 Relation to Supply Chains and Geopolitical Modeling

9.1 Mapping Supply Chains to MDPs

A supply chain can be formulated as an MDP:

Table 6: Supply Chain → MDP Mapping

Entity	Supply Chain	MDP
State	Inventory, demand forecast, supplier status	$s_t \in \mathcal{S}$
Action	Production quantity, procurement timing	$a_t \in \mathcal{A}$
Transition	Inventory dynamics, lead times, demand realization	$\mathcal{P}(s' \mid s, a)$
Reward	Profit = Revenue − Production − Holding − Stockout	$r(s, a, s')$
Objective	Maximize long-term profit	$\max \mathbb{E}[\sum \gamma^t r_t]$

9.1.1 State Space

$$s_t = \begin{bmatrix} I_t^{(\text{raw})} \\ I_t^{(\text{work})} \\ I_t^{(\text{finished})} \\ D_t \\ S_t^{(\text{supplier})} \end{bmatrix} \tag{26}$$

where:

- $I_t^{(\text{raw})}$ = raw material inventory
- $I_t^{(\text{work})}$ = work-in-process
- $I_t^{(\text{finished})}$ = finished goods
- D_t = demand signal (stochastic)
- $S_t^{(\text{supplier})}$ = supplier reliability index (0–1)

9.1.2 Action Space

$$a_t = \begin{bmatrix} q_t^{\text{order}} \\ t_{\text{expedite}}^{(\text{delivery})} \end{bmatrix} \tag{27}$$

where:

- q_t^{order} = order quantity (continuous, bounded)
- $t_{\text{expedite}}^{(\text{delivery})}$ = expedite flag (discrete: standard, expedited, switch supplier)

9.1.3 Stochastic Transitions

Demand is stochastic:

$$D_{t+1} \sim \text{AR}(1) + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma_D^2) \tag{28}$$

Lead times are stochastic:

$$L_t \sim \text{Geom}(p_{\text{delivery}}) + L_{\min} \tag{29}$$

Supplier reliability can change (exogenous shocks, geopolitical events):

$$S_{t+1} = S_t + \mathbb{1}(\text{shock}) \cdot (-\Delta S) \tag{30}$$

9.1.4 Reward Function

$$\begin{aligned} r_t &= \text{Revenue} - \text{Production} - \text{Holding} - \text{Stockout} && (31)\\ &= p_{\text{unit}} \cdot \min(D_t, I_t^{(\text{finished})}) - c_{\text{prod}} \cdot q_t^{(\text{prod})} && (32)\\ &\quad - h \cdot (I_t^{(\text{raw})} + I_t^{(\text{work})} + I_t^{(\text{finished})}) && (33)\\ &\quad - \text{penalty} \cdot \max(0, D_t - I_t^{(\text{finished})}) && (34) \end{aligned}$$

where the stockout penalty models lost sales, customer dissatisfaction, and reputational cost.

9.2 Agents as Firms / Countries

In a multi-agent setting:

- **Each agent**: A firm or nation managing supply chains
- **Shared resources**: Suppliers, transportation networks, raw materials
- **Competing objectives**: Each agent maximizes own profit
- **Non-stationarity**: Each agent's policy change affects others' environment

This is precisely the MARL setting: agents simultaneously optimize under mutual dependence.

9.3 Demand Shocks and Stochastic Transitions

Real supply chains experience:

1. **Seasonal demand**: Predictable variation (e.g., holiday demand spikes)
2. **Black swan shocks**: COVID lockdowns, geopolitical crises, natural disasters
3. **Supply disruptions**: Factory shutdowns, port congestion, transportation failures
4. **Price volatility**: Raw material costs fluctuate with commodity markets

An MDP with stochastic transitions captures this:

$$r_t = f(s_t, a_t, \varepsilon_t^{\text{demand}}, \varepsilon_t^{\text{supply}}, \varepsilon_t^{\text{geopolitical}}) \tag{35}$$

Value-based methods (Q-Learning) and policy gradient methods (PPO) both optimize expected reward despite these shocks.

9.4 Why MARL is Essential

9.4.1 Limitation of Single-Agent RL

Single-agent optimization assumes the environment is stationary or exogenous. In supply chains, other firms' actions affect:

- Supplier availability (if competitor orders heavily, lead times increase)
- Raw material prices (competing bids drive costs up)
- Transportation capacity (congestion from others' shipments)
- Market prices (competitor actions affect demand you face)

Single-agent RL learns policies against a fixed or randomly changing environment, missing strategic adaptation.

9.4.2 MARL Formulation

In MARL, each agent i maximizes:

$$J_i(\pi_1, \ldots, \pi_n) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_i(s_t, a_1^{(t)}, \ldots, a_n^{(t)})\right] \tag{36}$$

where $a_j^{(t)}$ is agent j's action at time t. Agents learn *best responses* to each other's policies, leading to Nash equilibria or other solution concepts.

9.4.3 Supply Chain Applications

- **Bullwhip effect**: Demand amplification upstream due to agents over-ordering in response to perceived scarcity
- **Price wars**: Competition drives margins down; MARL captures strategic pricing decisions
- **Supplier concentration**: When few suppliers exist, their decisions heavily impact buyers; MARL models this strategic interaction
- **Geopolitical resilience**: Nations optimize supply chains knowing competitors may do the same, leading to robust (or fragile) equilibria

10 Limitations and Future Work

10.1 Time Constraints

This project was completed during a 4-week winter program. Consequently:

- **Limited dataset scope**: Only two benchmark environments; real supply chains are vastly more complex
- **Shallow PPO exploration**: Custom environment is toy-scale; scaling to realistic dynamics (partial observability, high-dimensional states) is deferred
- **No multi-agent implementation**: Conceptual grounding provided, but true MARL systems require significantly more infrastructure

10.2 Limited Geopolitical Modeling

The supply chain–geopolitical mapping remains largely theoretical:

- No explicit game-theoretic analysis (Nash equilibria, Pareto frontiers)
- No calibration to real trade data or geopolitical indices
- No temporal dynamics of policy changes (tariffs, sanctions regimes)
- No heterogeneous agent types (small firms vs multinationals)

10.3 Scaling to True MARL Supply Chains

Practical MARL supply chain modeling would require:

1. **Partial Observability**: Agents don't observe others' inventory or policies. Leads to POMDPs and decentralized learning.
2. **Communication Protocols**: Whether and how agents communicate affects equilibrium efficiency.
3. **Complex Networks**: Real supply chains have thousands of firms and suppliers; scalable algorithms are needed.
4. **Hierarchical Control**: Global vs local decision-making; multi-timescale optimization.
5. **Adversarial Robustness**: Agents may be adversarial (competitors, bad actors); algorithms must be robust.

10.4 Future Directions

10.4.1 Short-Term (1–2 months)

- Implement multi-agent version of FrozenLake: two agents racing to goal, sharing grid
- Apply independent learners (IAQL) and compare to single-agent baseline
- Analyze convergence to pure/mixed strategy equilibria

10.4.2 Medium-Term (3–6 months)

- Develop realistic supply chain simulator with real demand/supply data
- Implement decentralized policy gradient (QMIX, MADDPG) to handle partial observability
- Compare MARL solutions to centralized planning (compute Pareto gap)

10.4.3 Long-Term (Publication-Grade Research)

- Game-theoretic analysis: Do agents converge to efficient equilibria?
- Adversarial robustness: Can agents defend against supply chain attacks?
- Real-world validation: Collaborate with industry partners on actual supply network optimization
- Policy synthesis: Generate human-interpretable rules from learned policies

11 Conclusion

This project provided rigorous grounding in reinforcement learning fundamentals and contemporary deep RL methods, with direct application to supply chain optimization and multi-agent systems.

11.1 Key Learnings

11.1.1 Theoretical Understanding

- **MDP formulation**: Enabled systematic modeling of stochastic environments
- **Value vs Policy methods**: Learned distinct trade-offs (sample efficiency, convergence guarantees, approximation errors)
- **On-policy vs Off-policy**: Visceral understanding of why Q-Learning outperforms SARSA in stochastic settings (48% vs 32% success)
- **Actor–Critic methods**: Integration of policy gradients with learned value function baselines enables continuous control
- **PPO stability**: Clipping and entropy regularization provide robust policy optimization without catastrophic updates

11.1.2 Practical Implementation

- Debugged complex RL codebases, understanding subtle issues (shape mismatches, log-probability arithmetic, distribution sampling)
- Implemented both tabular (Q-Learning, SARSA) and function approximation (PPO) algorithms from scratch
- Designed custom environments combining discrete and continuous actions
- Performed systematic hyperparameter sweeps to validate robustness

11.1.3 Application Insight

- Mapped real-world supply chains and geopolitical systems onto MDPs, clarifying where RL applies
- Identified key drivers of single-agent vs multi-agent complexity
- Demonstrated why stochastic transitions require careful algorithm choice (off-policy better for optimistic exploration)

11.2 Practical Implications

11.2.1 Supply Chain Optimization

Firms can deploy RL agents to:

- Optimize inventory levels under stochastic demand and lead times
- Dynamically switch suppliers based on reliability and price
- Learn robust policies that perform well despite demand uncertainty

The 48% vs 32% performance gap between Q-Learning and SARSA on FrozenLake translates to potentially significant profit improvements in real supply chains, justifying investment in RL tools.

11.2.2 Strategic Decision-Making

MARL frameworks enable nations/firms to:

- Model strategic interactions in trade networks
- Evaluate impact of tariffs, sanctions, or investment on equilibrium outcomes
- Design robust supply chains that withstand adversarial actions

11.3 Research Readiness

This project demonstrates:

1. **Mastery of foundational theory**: Bellman equations, convergence proofs, function approximation
2. **Algorithm implementation**: Ability to code non-trivial RL methods end-to-end
3. **Experimental rigor**: Hyperparameter sweeps, ablation studies, statistical validation
4. **Problem formulation**: Recognition of when RL is appropriate and how to map applications
5. **Scalability mindset**: Understanding bottlenecks and pathways to real-world deployment

These competencies form the foundation for:

- Advanced RL research (inverse RL, meta-RL, hierarchical control)
- Industry applications (trading algorithms, autonomous systems, resource optimization)
- Cross-disciplinary work (game theory, mechanism design, controls)

11.4 Final Remarks

Reinforcement learning provides a powerful lens for understanding decision-making under uncertainty. From classical value iteration to modern policy gradients, the field has developed elegant algorithms with proven convergence guarantees and impressive empirical performance. The journey from tabular Q-Learning (48% success) to PPO for continuous control demonstrates the field's maturity and breadth.

The mapping to supply chains and geopolitics reveals RL's relevance beyond robotics and gaming. As systems become more complex and decisions more consequential, principled approaches to optimization under stochasticity become indispensable.

Future work will deepen this exploration, extending from single-agent foundations to full multi-agent systems where strategic interaction and learning occur simultaneously. The WiDS program has provided an excellent launchpad for this journey.

References

[1] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

[2] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Openai, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

[3] Mnih, V., Kavukcuoglu, K., & Silver, D. (2013). Playing Atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*.

[4] Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). OpenAI Gym. *arXiv preprint arXiv:1606.01540*.

[5] Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2015). High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*.

[6] Foerster, J., Assael, Y. M., de Freitas, N., & Whiteson, S. (2017). Learning to communicate with deep multi-agent reinforcement learning. *arXiv preprint arXiv:1605.06676*.

[7] Watkins, C. J., & Dayan, P. (1992). Q-learning. *Machine Learning*, *8*(3-4), 279-292.

[8] Rummery, G. A., & Niranjan, M. (1994). On-line Q-learning using connectionist systems (Vol. 37). University of Cambridge Department of Engineering.

[9] Buşoniu, L., Babuška, R., & De Schutter, B. (2010). Multi-agent reinforcement learning: An overview. In *Innovations in multi-agent systems and applications-1* (pp. 183-221). Springer, Berlin, Heidelberg.

[10] Littman, M. L. (1994). Markov games as a framework for multi-agent reinforcement learning. In *Proceedings of the eleventh international conference on machine learning* (pp. 157-163).

[11] Ivgi, M., Cohen, A., & Amos, B. (2023). Return-based scaling for offline reinforcement learning. In *Advances in Neural Information Processing Systems* (pp. 1-17).

[12] Lowe, R., Wu, Y. I., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments. In *International conference on machine learning* (pp. 2094-2103). PMLR.